CORE103 Computational Thinking
PW5 - Nested Iteration



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.1.1 Lab Objectives 2

2 Prerequisites **3**
2.1 Assert Statements 3
2.2 Pytest 3

3 Exercises **5**
3.1 Arcade Combo Multiplier Display 5
3.2 Arcade Power-Level Diagnostic 6
3.3 Festival LED Channel Numbering 8
3.4 Scoreboard Alignment Test 9
3.5 Inverted Number Display Test 10
3.6 Stage LED Diamond Preview 11
3.7 Campus Event Pyramid Debugging 13
3.8 Team Code Permutation Challenge 15
3.9 Prime ID Lucky Draw 16
3.10 Campus Fest Lucky Code Challenge 17

Introduction

1.1 Instructions

- This PW must be submitted online via CANVAS. You are required to submit a zip file that contains all the `.py` files.
- The zip file should be named as `PW5_aa123456.zip` where `aa123456` will be replaced with your student id.

1.1.1 Lab Objectives

The following are the lab objectives of this lab:

- Nested Iteration in Python

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, 'x should be a non-negative number'
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

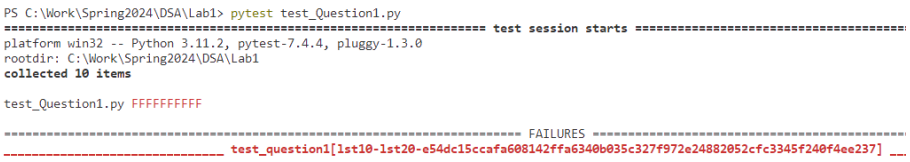
If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

============================== 10 passed in 0.06s =========
```

However, if one or more tests fail, you should receive a similar output.



Exercises

3.1 Arcade Combo Multiplier Display

This question should be implemented in the file ***q1.py****.*

Before a campus game night begins, an arcade machine displays the multiplier progression for each combo level based on a given positive integer level_count.

For example, if level_count is `5`, the display should show:

```
5 25 125 625 3125
4 16 64 256
3 9 27
2 4
1
```

Given level_count, print the complete combo pattern.

Constraints

- `isinstance(level_count, int)` is `True`.
- $1 \leq \textit{level_count} \leq 10$
- Use **nested iteration**.
- Do not hardcode the output.

Function Specification

Write an **iterative** function named **print_combo_multipliers** that takes one parameter, **level_count**, and prints the pattern illustrated above.

The function must print exactly level_count rows. Numbers appearing on the same row must be separated by one space. The arrangement of the values must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> print_combo_multipliers(5)
5 25 125 625 3125
4 16 64 256
3 9 27
2 4
1

>>> print_combo_multipliers(6)
6 36 216 1296 7776 46656
5 25 125 625 3125
4 16 64 256
3 9 27
2 4
1
```

Testing

To test your function, type:

```
pytest tests/test_q1.py
```

3.2 Arcade Power-Level Diagnostic

This question should be implemented in the file ***`q2.py`***.

Before an arcade game is opened to players, a technician runs a diagnostic display that shows the game's power levels based on a given positive integer row_count.

For example, if row_count is `5`, the display should show:

```
1
2 1
4 2 1
8 4 2 1
16 8 4 2 1
```

Given row_count, print the complete power pattern.

Constraints

- `isinstance(row_count, int)` is `True`.
- $row_count \geq 1$
- Use **nested iteration**.
- Do not hardcode the output.

Function Specification

Write an **iterative** function named **print_power_levels** that takes one parameter, **row_count**, and prints the pattern illustrated above.

The function must print exactly row_count rows. Numbers appearing on the same row must be separated by one space. The arrangement of the values must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> print_power_levels(3)
1
2 1
4 2 1

>>> print_power_levels(1)
1

>>> print_power_levels(2)
1
2 1

>>> print_power_levels(6)
1
2 1
4 2 1
8 4 2 1
16 8 4 2 1
32 16 8 4 2 1

>>> print_power_levels(8)
1
2 1
4 2 1
8 4 2 1
16 8 4 2 1
32 16 8 4 2 1
64 32 16 8 4 2 1
128 64 32 16 8 4 2 1
```

Testing

To test your function, type:

```
pytest tests/test_q2.py
```

3.3 Festival LED Channel Numbering

This question should be implemented in the file `q3.py`.

During festival setup, a technician assigns consecutive channel numbers to rows of an LED panel. The preview is generated for a given positive integer row_count.

For example, if row_count is `3`, the display should show:

```
1
2 3 4
5 6 7 8 9
```

Given row_count, print the complete number pattern.

Constraints

- `isinstance(row_count, int)` is `True`.
- $row_count \geq 1$
- Use **nested iteration**.
- Do not hardcode the output.

Function Specification

Write an **iterative** function named **print_led_channels** that takes one parameter, **row_count**, and prints the pattern illustrated above.

The function must print exactly row_count rows. Numbers appearing on the same row must be separated by one space. The arrangement and continuation of the numbers must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> print_led_channels(3)
1
2 3 4
5 6 7 8 9

>>> print_led_channels(1)
1

>>> print_led_channels(6)
1
2 3 4
5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23 24 25
26 27 28 29 30 31 32 33 34 35 36
```

Testing

To test your function, type:

```
pytest tests/test_q3.py
```

3.4 Scoreboard Alignment Test

This question should be implemented in the file ***q4.py****.*

Before a campus game night, a technician tests the alignment of a digital scoreboard using a number display based on a given positive integer max_number.

For example, if max_number is `5`, the scoreboard displays:

```
0 1 2 3 4 5
  0 1 2 3 4
    0 1 2 3
      0 1 2
        0 1
          0
```

Given max_number, print the complete scoreboard pattern.

Constraints

- `isinstance(max_number, int)` is `True`.
- $1 \leq max_number \leq 9$
- Use **nested iteration**.
- Do not hardcode the output.
- Do not use string multiplication to generate repeated spaces.

Function Specification

Write an **iterative** function named **print_scoreboard_alignment** that takes one parameter, **max_number**, and prints the pattern illustrated above.

The function must print exactly max_number rows. Numbers appearing on the same row must be separated by one space. The placement of the leading spaces and the arrangement of the numbers must follow the pattern shown in the sample interaction. Leading spaces are part of the required output.

Sample Interaction

```
>>> print_scoreboard_alignment(5)
0 1 2 3 4 5
  0 1 2 3 4
    0 1 2 3
      0 1 2
        0 1

>>> print_scoreboard_alignment(1)
0 1

>>> print_scoreboard_alignment(2)
0 1 2
  0 1
```

Testing

To test your function, type:

```
pytest tests/test_q4.py
```

3.5 Inverted Number Display Test

This question should be implemented in the file ***q5.py****.*

During festival setup, a technician tests a console display using an inverted number layout based on a given positive integer row_count.

For example, if row_count is `5`, the display should show:

```
5 5 5 5 5
 4 4 4 4
  3 3 3
   2 2
    1
```

Given row_count, print the complete pattern.

Constraints

- `isinstance(row_count, int)` is `True`.
- $1 \leq \textit{row_count} \leq 9$
- Use **nested iteration**.
- Do not hardcode the output.
- Do not use string multiplication to generate repeated spaces or values.

Function Specification

Write an **iterative** function named **print_inverted_number_triangle** that takes one parameter, **row_count**, and prints the pattern illustrated above.

The function must print exactly row_count rows. Numbers appearing on the same row must be separated by one space. The placement of the leading spaces and the arrangement of the numbers must follow the pattern shown in the sample interaction. Leading spaces are part of the required output.

Sample Interaction

```
>>> print_inverted_number_triangle(5)
5 5 5 5 5
 4 4 4 4
  3 3 3
   2 2
    1

>>> print_inverted_number_triangle(3)
3 3 3
 2 2
  1

>>> print_inverted_number_triangle(1)
1
```

Testing

To test your function, type:

```
pytest tests/test_q5.py
```

3.6 Stage LED Diamond Preview

This question should be implemented in the file ***q6.py****.*

Before the university festival, the stage team uses a console preview to test a diamond-shaped LED layout.

For example, a diamond of size `5` is:

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

The upper and lower parts of the diamond must be handled using separate helper functions.

Constraints

- `isinstance(size, int)` is `True`.
- $1 \leq size \leq 10$
- Use **nested iteration**.
- Do not hardcode the output.
- Do not use string multiplication to generate repeated spaces or asterisks.

Function Specification

Your solution must contain the following three functions:

- **print_upper_diamond(size)** – prints the upper half, including the middle row.
- **print_lower_diamond(size)** – prints the lower half, excluding the middle row.
- **print_diamond(size)** – calls both helper functions to print the complete diamond.

print_upper_diamond(size) must print exactly size rows. **print_lower_diamond(size)** must print exactly size -1 rows. Both helpers use the same alignment as the complete diamond.

The complete diamond must contain $2 \times size - 1$ rows. Asterisks on each row must be consecutive, with no spaces between them. Use leading spaces to center the pattern.

Helper Function Output

For size equal to 3, the helper functions must print exactly:

```
>>> print_upper_diamond(3)
  *
 ***
*****

>>> print_lower_diamond(3)
 ***
  *
```

Sample Interaction

```
>>> print_diamond(5)
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *

>>> print_diamond(4)
   *
  ***
 *****
*******
 *****
  ***
   *
```

Testing

To test your functions, type:

```
pytest tests/test_q6.py
```

3.7 Campus Event Pyramid Debugging

This question should be implemented in the file ***q7.py****.*

A campus event display should print a pyramid of a given height height, but the provided implementation contains errors.

For example, if height is 5, the display should produce:

```
    *
   * *
  * * *
 * * * *
* * * * *
```

The file `q7.py` contains the complete buggy implementation, including **print_pyramid** and its helper functions. Debug the provided code so that the required pyramid is printed correctly.

Constraints

- `isinstance(height, int)` is `True`.
- $1 \leq \textit{height} \leq 40$
- Use **nested iteration**.
- Do not change the names or parameters of the provided functions.
- Do not remove the provided helper functions.
- Do not use string multiplication to generate repeated spaces or asterisks.

Function Specification

The function **print_pyramid** takes one parameter, **height**. Correct the provided implementation so that it prints the pyramid illustrated above.

The function must print exactly height rows. The placement of spaces and asterisks must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> print_pyramid(5)
    *
   * *
  * * *
 * * * *
* * * * *

>>> print_pyramid(3)
  *
 * *
* * *
```

Testing

To test your function, type:

```
pytest tests/test_q7.py
```

3.8 Team Code Permutation Challenge

This question should be implemented in the file ***q8.py****.*

As a warm-up challenge before a campus coding competition, each team generates all arrangements of a three-letter team code containing three unique letters.

For example, for the team code `ABC`, the possible arrangements are:

```
ABC ACB BAC BCA CAB CBA
```

Given team_code, print all possible arrangements of its letters.

Constraints

- `isinstance(team_code, str)` is `True`.
- `len(team_code) == 3`
- `team_code.isalpha()` is `True`.
- All three letters in team_code are unique.
- Use **nested iteration**.
- Do not use `itertools.permutations()` or any library function that directly generates permutations.

Function Specification

Write an **iterative** function named **print_code_permutations** that takes one parameter, **team_code**, and prints all possible arrangements of its letters.

Print all six arrangements on **one line**, separated by one space. The order in which the arrangements are printed does not matter.

Sample Interaction

```
>>> print_code_permutations('ABC')
ABC ACB BAC BCA CAB CBA

>>> print_code_permutations('XYA')
XYA XAY YXA YAX AXY AYX
```

Testing

To test your function, type:

```
pytest tests/test_q8.py
```

3.9 Prime ID Lucky Draw

This question should be implemented in the file ***q9.py****.*

For a campus lucky draw, the organizers choose prime-numbered entry IDs as special IDs that receive an extra entry.

A prime number is an integer greater than `1` that has exactly two positive divisors: `1` and itself.

Given start_id and end_id, print all prime-numbered IDs in the range, including both endpoints.

Constraints

- `isinstance(start_id, int)` and `isinstance(end_id, int)` are `True`.
- $0 \leq start_id \leq end_id \leq 10^5$
- You may assume that all test cases use ranges that can be processed within a reasonable amount of time.
- Use **nested iteration**.
- Do not use a library function that directly determines or generates prime numbers.

Function Specification

Write an **iterative** function named **print_prime_ids** that takes two parameters, **start_id** and **end_id**, and prints all prime-numbered IDs in the given range.

Print the IDs in increasing order on **one line**, separated by a comma followed by one space. Do not print a trailing comma after the final number. If the range contains no prime numbers, do not print anything.

Sample Interaction

```
>>> print_prime_ids(1, 10)
2, 3, 5, 7

>>> print_prime_ids(3, 5)
3, 5

>>> print_prime_ids(20, 30)
23, 29

>>> print_prime_ids(14, 16)
```

Testing

To test your function, type:

```
pytest tests/test_q9.py
```

3.10 Campus Fest Lucky Code Challenge

This question should be implemented in the file ***q10.py****.*

As a final festival puzzle, students check whether a six-digit entry code qualifies as a **Lucky Code**.

A six-digit entry code is a Lucky Code if it is already a palindrome, or if changing **at most one digit** can make it a palindrome. A palindrome reads the same forwards and backwards.

For example, `218812` is already a palindrome, while `213212` can become the palindrome `213312` by changing one digit. Both are Lucky Codes.

Given entry_code, determine whether it is a Lucky Code.

Constraints

- `isinstance(entry_code, int)` is `True`.
- $100000 \leq \textit{entry_code} \leq 999999$
- Use **iteration**.
- Do not convert entry_code to a string.
- Do not use string slicing, `reversed()`, lists, or other shortcuts that directly compare the digits as a sequence.

Function Specification

Write an **iterative** function named **is_lucky_code** that takes one parameter, **entry_code**.

Return `"YES"` if the entry code is a Lucky Code. Otherwise, return `"NO"`.

Sample Interaction

```
>>> is_lucky_code(213212)
'YES'

>>> is_lucky_code(218812)
'YES'

>>> is_lucky_code(121212)
'NO'
```

Testing

To test your function, type:

```
pytest tests/test_q10.py
```